Dpto. de Ingeniería de Sistemas y Computación

Asignatura: Diseño de Software I

Documento de Especificación de Requisitos de Software

TeamMatch Plataforma de matchmaking para equipos de trabajo

Versión 1.0

Integrantes:

Andrea Carolina de la Ossa Gelves.

Daniel Eduardo Cera Moran.

Nicolas Pérez Padilla.

Jhosep Varela Regalado.

Iván Rueda.

Darlen Cecilia Arias Alemán.

Docente: Daniel José Romero Martínez

10 de abril del 2026

CONTEXTO DEL PROBLEMA..... 2
Descripción del proyecto..... 2
Objetivo General. 2
Objetivos específicos..... 3
Supuestos del Proyecto. 4
Restricciones. 5
Justificación..... 7
ACTORES DEL SISTEMA. 7
1. Usuario (Estudiante / Profesional) 7
2. Sistema de Matchmaking (Sistema Inteligente) 8
4. Proyecto (Entidad organizadora)..... 8
5. Líder de equipo (rol derivado)..... 8
EQUIPO DEL PROYECTO. 9
REQUERIMIENTOS. 9
Requerimientos Funcionales. 9
Requerimientos No Funcionales..... 11
ESTRUCTURA DEL SISTEMA..... 11
Componentes del sistema. 12
Relación entre componentes. 13
Diseño del modelo de datos. 13
Diagrama de base de datos. 14
DIVISIÓN POR MÓDULOS..... 14
Primer Diagrama de Actividad 16
Segundo Diagrama de Actividad: Gestión de tareas 17
ARTEFACTOS OBLIGATORIOS. 18
REFERENCIAS. 18

CONTEXTO DEL PROBLEMA.

Descripción del proyecto.

TeamMatch es una plataforma digital diseñada para facilitar la formación de equipos de trabajo de manera inteligente y eficiente. El sistema permite a los usuarios registrar sus habilidades, experiencia y disponibilidad en lenguaje natural o mediante la carga de documentos (pdf o texto plano), los cuales son procesados automáticamente por un modelo de lenguaje (SLM/LLM) que extrae, normaliza y agrupa dichos componentes antes de persistirlas en Neo4j. Adicionalmente, el motor de emparejamiento aplica las reglas de negocio a través de un modelo de IA, con el fin de generar automáticamente equipos equilibrados y compatibles.

A diferencia de los métodos tradicionales de selección manual, TeamMatch combina reglas de negocio con capacidades de inteligencia artificial para analizar la información de los usuarios y proponer combinaciones óptimas, teniendo en cuenta factores como complementariedad de habilidades, afinidad entre perfiles y necesidades específicas de cada proyecto

Además, la plataforma permite gestionar la interacción entre los miembros del equipo, facilitando la asignación de roles, el seguimiento del desempeño y la retroalimentación, con el objetivo de mejorar continuamente la calidad de los equipos formados.

Objetivo General.

Desarrollar un sistema de software que permita la formación automática de equipos de trabajo mediante el análisis de habilidades, experiencia y disponibilidad descritas en lenguaje natural, procesadas por un modelo SLM/LLM para su normalización y agrupación en Neo4j, y cuyo emparejamiento sea resuelto mediante un modelo de IA que aplique las reglas de negocio definidas, con el fin de optimizar la colaboración, mejorar la eficiencia en la asignación de roles y garantizar una mayor compatibilidad entre los integrantes.

Objetivos específicos.

1. Diseñar e implementar un módulo de registro y gestión de perfiles de usuario
Desarrollar un sistema de autenticación y gestión de cuentas que permita a los usuarios registrar y actualizar su información personal. Las habilidades, experiencia y disponibilidad se ingresarán mediante texto en lenguaje natural o la carga de archivos pdf (o texto plano). Un modelo SLM/LLM procesará estos datos para extraer,

normalizar y agrupar dichos componentes, almacenándolos estructuradamente en Neo4j como base para el proceso de formación de equipos.

2. Desarrollar un motor de emparejamiento basado en IA y reglas de negocio Implementar un motor que utilice un modelo de IA para aplicar las reglas de negocio definidas sobre el grafo de Neo4j, analizando los perfiles procesados para generar automáticamente propuestas de equipos equilibrados. El modelo considerará criterios como complementariedad de habilidades normalizadas, compatibilidad de disponibilidad, experiencia y afinidad entre perfiles.

3. Implementar un módulo de gestión de proyectos y definición de requerimientos de equipo Crear una funcionalidad que permita a los organizadores o líderes definir las características del proyecto, los roles requeridos y las habilidades necesarias, de modo que el sistema pueda filtrar y proponer los perfiles más adecuados según las necesidades específicas de cada iniciativa.

4. Desarrollar un módulo de asignación de roles y seguimiento de desempeño Construir una interfaz que facilite la asignación formal de roles dentro del equipo formado, el registro de tareas o responsabilidades por integrante y el seguimiento básico del cumplimiento, permitiendo a los líderes tener visibilidad sobre la participación de cada miembro.

5. Integrar un sistema de retroalimentación entre miembros del equipo Implementar un mecanismo de evaluación mutua al cierre de un proyecto o ciclo de trabajo, que permita a los integrantes valorar la colaboración, la comunicación y el desempeño de sus compañeros, con el fin de enriquecer los perfiles y mejorar la precisión del motor de emparejamiento en futuras asignaciones.

6. Diseñar e implementar la arquitectura del sistema aplicando principios de Diseño de Software Definir y documentar la arquitectura de la plataforma utilizando patrones de diseño apropiados (MVC, capas, repositorio, entre otros), diagramas UML y buenas prácticas de ingeniería de software, garantizando que el sistema sea escalable, mantenible y coherente con los fundamentos de la asignatura.

Supuestos del Proyecto.

Del entorno y los usuarios

1. Los usuarios (estudiantes y profesionales) cuentan con acceso a internet y un dispositivo con navegador web moderno para interactuar con la plataforma.

2. Los usuarios están dispuestos a registrar su información de habilidades, experiencia y disponibilidad de manera honesta y completa, dado que la calidad del emparejamiento depende directamente de estos datos.
3. Existe una base mínima de usuarios registrados con perfiles diversos para que el motor de emparejamiento pueda generar equipos con variedad de habilidades y roles.
4. Los usuarios comprenden el idioma en que está desarrollada la interfaz de la plataforma.

Del equipo de desarrollo

5. Los seis integrantes del equipo tienen conocimientos básicos en TypeScript, React y Node.js, o la disposición de adquirirlos durante el desarrollo del proyecto.
6. El equipo utiliza GitHub como repositorio central de código, con un flujo de trabajo basado en ramas (feature branches) y pull requests para controlar la calidad del código.
7. Jira es la herramienta oficial de gestión de tareas del proyecto, y todos los integrantes se comprometen a mantener el tablero actualizado durante los sprints.
8. Enterprise Architect se utiliza para producir y mantener la documentación de diseño (diagramas UML), y todos los integrantes tienen acceso a una licencia válida de la herramienta.
9. Las decisiones de arquitectura y diseño deben estar documentadas en Enterprise Architect y figma antes de iniciar la implementación de cada módulo.

De la tecnología

10. El IDE del proyecto será VS CODE
11. Neo4j es adecuado para modelar y consultar las relaciones entre perfiles de usuario, habilidades normalizadas y equipos.
12. Se dispondrá de acceso a una API de un modelo SLM/LLM (Gemini) para el procesamiento de habilidades en lenguaje natural y para la aplicación de reglas de negocio en el motor de emparejamiento.
13. Docker garantiza que todos los integrantes del equipo trabajen en entornos de desarrollo idénticos, eliminando inconsistencias entre máquinas locales.
14. Los contenedores Docker definidos para el frontend (React + Vite), el backend (Node.js + Express) y la base de datos (Neo4j) y el servicio de procesamiento IA pueden ejecutarse de forma conjunta en las máquinas del equipo sin problemas de recursos.
15. Las librerías y dependencias de npm utilizadas en el proyecto son compatibles con TypeScript y no presentarán cambios de licencia o deprecación durante el ciclo de desarrollo.
16. JWT es suficiente como mecanismo de autenticación y autorización para el alcance definido del proyecto, sin requerir un proveedor de identidad externo.

17. Los prototipos de interfaz de usuario se diseñarán en Figma antes de iniciar el desarrollo del frontend, sirviendo como referencia visual vinculante para el equipo de desarrollo.

Del alcance y el negocio

18. Las reglas de negocio del motor de emparejamiento se definen, revisan y aprueban con el docente antes de comenzar su implementación en Neo4j.
19. El proyecto no requiere integraciones con sistemas externos (herramientas de RRHH) en esta primera versión.
20. El volumen de usuarios simultáneos durante las pruebas es reducido, por lo que no se contempla optimización de rendimiento a escala en esta etapa del proyecto.
21. El despliegue final de la aplicación se realiza en un entorno local o de laboratorio mediante Docker Compose, sin requerir infraestructura cloud en producción.

Restricciones.

De tiempo

1. El proyecto debe desarrollarse dentro del período académico establecido por la asignatura, sin posibilidad de extender el plazo de entrega.
2. Los sprints de desarrollo deben ajustarse al calendario académico, incluyendo fechas de parciales y otras entregas que reducen la disponibilidad del equipo.
3. Riesgo: los retrasos por carga académica acumulada pueden comprometer entregas parciales. Se mitiga planificando sprints con holgura e identificando semanas críticas desde el inicio.

Del equipo

4. El equipo está conformado exclusivamente por seis integrantes; no es posible incorporar personas adicionales ni contratar servicios externos de desarrollo.
5. Las decisiones técnicas deben ser tomadas por el equipo sin depender de expertos externos, salvo la orientación del docente.

De tecnología

6. El stack tecnológico está fijado desde el inicio (React + Vite, Node.js + Express, Figma, TypeScript, Neo4j, Docker, GitHub, Jira, Enterprise Architect, API LLM/SLM) y no puede modificarse durante el desarrollo.
7. El sistema debe ejecutarse correctamente mediante Docker Compose en equipos con hardware estándar, sin requerir servidores dedicados.
8. No se contempla el uso de servicios cloud de pago; todos los entornos son locales o gratuitos.

9. Riesgo: inconsistencias entre entornos de desarrollo a pesar del uso de Docker. Se mitiga versionando el `docker-compose.yml` desde el inicio y documentando el proceso de setup en el README del repositorio.

De alcance

10. El sistema no incluye funcionalidades de mensajería en tiempo real entre usuarios en esta versión.
11. La plataforma no contempla integración con herramientas externas de gestión de proyectos desde la aplicación misma.
13. El sistema no procesará datos financieros ni gestionará pagos de ningún tipo.
14. Riesgo: el alcance del proyecto puede crecer de forma no controlada (scope creep) a medida que se identifican nuevas funcionalidades deseables. Se mitiga congelando el alcance en Jira al inicio de cada sprint y gestionando cualquier cambio mediante un proceso formal de aprobación con el docente.

De diseño y arquitectura

15. La arquitectura del sistema y los diagramas UML deben estar definidos y aprobados en Enterprise Architect antes de iniciar la implementación de cada módulo.
16. Riesgo: falta de acuerdo temprano en las reglas de negocio del motor de emparejamiento puede generar reproceso en el modelo de grafos. Se mitiga validando las reglas con el docente antes de implementarlas y documentándolas formalmente como parte de los requisitos del sistema.

Justificación.

La formación de equipos de trabajo es una actividad crítica en entornos académicos y empresariales que, en la mayoría de los casos, se realiza de forma subjetiva e informal. Criterios como la afinidad personal o la simple disponibilidad generan con frecuencia equipos desequilibrados, con habilidades redundantes y roles mal asignados, afectando directamente la productividad y la calidad de los resultados.

TeamMatch surge como respuesta a esta problemática, ofreciendo una plataforma que automatiza y objetiva el proceso de formación de equipos mediante el análisis de perfiles de usuario. Su principal diferenciador tecnológico es el uso de Neo4j, una base de datos de grafos que permite modelar de forma natural las relaciones entre personas, habilidades y proyectos; por otro lado, la incorporación de modelos SLM/LLM que permite procesar habilidades expresadas en lenguaje natural y aplicar reglas de negocio de emparejamiento de manera inteligente y flexible, algo que los modelos relacionales tradicionales no resuelven con la misma fluidez.

Los beneficios de la plataforma son concretos en dos frentes: en universidades, facilita la conformación de grupos de trabajo más diversos y complementarios; en empresas,

apoya a líderes y áreas de talento humano en la asignación eficiente de personas a proyectos. En ambos casos, TeamMatch transforma un proceso intuitivo en uno basado en datos, reduciendo tiempos de selección y mejorando la compatibilidad entre integrantes.

ACTORES DEL SISTEMA.

1. Usuario (Estudiante / Profesional)

Descripción:
Es la persona que utiliza la plataforma para formar parte de un equipo de trabajo.

Responsabilidades:

- Registrar y mantener actualizado su perfil, habilidades y disponibilidad horaria.
- Visualizar propuestas de equipos generadas por el sistema.
- Gestionar y marcar como finalizadas las tareas asignadas dentro de un proyecto.
- Proveer retroalimentación honesta sobre sus compañeros al finalizar un proyecto.

2. Sistema de Matchmaking (Sistema de IA)

Descripción:
Es el componente central encargado de procesar habilidades en lenguaje natural mediante un modelo SLM/LLM y de aplicar las reglas de negocio a través de IA para generar equipos de trabajo de manera automática.

Responsabilidades:

- Recibir texto libre o documentos adjuntos (pdf/txt) con habilidades, experiencia y disponibilidad del usuario y procesarlos mediante un modelo LLM/SLM para extraer, normalizar y agrupar dichas habilidades.
- Persistir las habilidades normalizadas como nodos y relaciones en el grafo Neo4j.
- Aplicar las reglas de negocio de emparejamiento a través de un modelo de IA que consulta el grafo y calcula el Score de afinidad.
- Generar propuestas de equipos equilibrados automáticamente.
- Garantizar de forma automatizada la disponibilidad del sistema y la integridad de los datos en el grafo.

3. Proyecto (Entidad organizadora)

Descripción:
Representa una actividad, reto o trabajo para el cual se requiere formar un equipo.

Responsabilidades:

- Almacenar la descripción, objetivos y roles requeridos.
- Servir como el nodo central de unión para los miembros asignados y las tareas.

4. Líder de equipo (rol derivado)

Descripción:
Es un usuario que asume la responsabilidad de coordinar un equipo dentro de un proyecto.

Responsabilidades:

- Definir requerimientos de equipo y roles necesarios.
- Asignar tareas, supervisar el progreso y validar o rechazar entregas.
- Modificar la composición del equipo si surgen imprevistos.
- Gestionar las cuentas y accesos de los usuarios integrantes de su proyecto, resolviendo incidencias de seguridad o permisos que correspondan a su equipo de trabajo.

EQUIPO DEL PROYECTO.

Nombre	Rol	Código	Nick de Discord
Darlen Cecilia Arias Alemán	DevOps y lead Developer	200182800	Darlena
Andrea Carolina de la Ossa Gelves	Frontend Engineering	200192335	Andrea
Daniel Eduardo Cera Moran	Database Administration (DBA)	200195354	dn
Nicolas Pérez Padilla	QA & Testing y SCRUM Master	200196735	nicolas
Jhosep Varela Regalado	Backend Engineering	200198584	Jhosep Varela
Iván Rueda	Backend Engineering:	200182491	ElRepollo05

REQUERIMIENTOS.

Requerimientos Funcionales.

RF1. Registro de usuario. El sistema debe permitir a un nuevo usuario crear una cuenta proporcionando nombre completo, correo electrónico y contraseña. El sistema debe validar que el correo no esté previamente registrado y que la contraseña cumpla con los criterios mínimos de seguridad definidos (mínimo 8 caracteres, al menos una mayúscula y un número).

RF2. Autenticación de usuario. El sistema debe permitir al usuario iniciar sesión con su correo y contraseña. Tras una autenticación exitosa, el sistema debe generar y retornar un token JWT con tiempo de expiración definido, el cual será requerido para acceder a los recursos protegidos de la plataforma.

RF3. Gestión de habilidades en el perfil. El sistema debe permitir al usuario registrar sus habilidades, experiencia y disponibilidad de dos formas: (a) mediante texto libre en lenguaje natural, describiendo sus competencias, trayectoria profesional y disponibilidad horaria con sus propias palabras; o (b) cargando un archivo adjunto en formato PDF o texto plano (.txt) que contenga información relevante, como un CV o descripción personal.

En ambos casos, el sistema enviará el contenido a un modelo SLM/LLM que se encargará de extraer y estructurar automáticamente tres bloques de información: las habilidades identificadas (normalizadas y agrupadas en técnicas, blandas y de dominio, con nivel de dominio básico, intermedio o avanzado cuando sea inferible), la experiencia (años de experiencia general o por habilidad y roles desempeñados), y la disponibilidad horaria (días de la semana y franjas horarias disponibles para participar en proyectos).

El usuario podrá revisar, corregir y eliminar cualquier dato extraído en los tres bloques antes de confirmar su guardado.

RF4. Registro de disponibilidad horaria. El sistema debe permitir al usuario actualizar su disponibilidad semanal indicando los días y franjas horarias en las que puede participar activamente en un proyecto.

RF5. Visualización de perfil público. El sistema debe permitir que otros usuarios registrados consulten el perfil de un usuario, visualizando sus habilidades, nivel de dominio, experiencia e intereses. La información de contacto personal (correo, contraseña) no debe ser visible en esta vista pública.

RF6. Generación Automática de Equipos. El sistema debe ejecutar un motor de emparejamiento basado en IA que procese los datos del grafo Neo4j y aplique las reglas de negocio definidas (complementariedad de habilidades normalizadas, compatibilidad horaria, experiencia e historial de retroalimentación) utilizando un modelo de lenguaje o indiferencia para evaluar candidatos y generar propuestas de equipos. El modelo de IA recibirá como contexto los perfiles de los candidatos, los requerimientos del proyecto y las reglas de negocio, y retornará las combinaciones óptimas de equipo con su justificación.

RF7. Visualización de Score de Afinidad. La aplicación debe mostrar un indicador de "afinidad" o "compatibilidad" para cada propuesta de equipo, justificando por qué el algoritmo seleccionó a esos miembros, incluyendo los criterios específicos evaluados por el modelo IA.

RF8. Definición de Requerimientos de Proyecto. El líder del proyecto debe poder crear un proyecto asignándole nombre, descripción, objetivos, y una lista de roles necesarios con sus respectivas habilidades requeridas.

RF9. Filtrado de Candidatos. El sistema debe permitir a los lideres realizar búsquedas manuales de perfiles utilizando filtros por etiquetas de habilidades, experiencia y disponibilidad (normalizadas por el LLM)

RF10. Tablero de Asignación de Tareas. Una vez formado el equipo, la plataforma debe permitir asignar tareas a uno o más miembros del equipo y asignar estados de cumplimiento a cada uno como "Pendiente", "En Proceso", “En Revisión” y "Completado".

RF11. Modificación del Equipo. El sistema debe permitir al líder del proyecto modificar los roles del equipo, hacia que miembro se dirigen ciertas asignaciones, o a los mismos integrantes. Esto siempre que ocurra alguna situación de inconformidad, inseguridad, o cualquier situación externa que pueda alterar el flujo de trabajo.

RF12. Llamado a Revisión de Asignación. Luego de finalizar una tarea particular, el sistema debe permitir, a cada miembro, marcarla como finalizada para que el sistema notifiqué al líder del proyecto para su revisión (cambia a estado “En Revisión”).

RF13. Verificación de Asignación. El sistema debe permitir al líder del proyecto validar (cambiar a estado “Completado”) o rechazar (volver al estado “En Proceso”) la culminación de dicha asignación según su consideración.

RF14. Sistema de Retroalimentación. Una vez culminado un proyecto, el sistema debe permitir realizar retroalimentaciones sobre la participación de los miembros de este mutuamente.

RF15. Historial de Trabajos. Una vez finalizado el sistema de retroalimentación de un proyecto, el sistema debe permitir visualizar dichas retroalimentaciones relacionadas a un miembro en particular, separadas por proyectos, en el mismo perfil principal de este usuario y que estos datos puedan ser utilizados por el motor de emparejamiento con IA.

Requerimientos No Funcionales

RNF1. Tiempo de respuesta del motor de emparejamiento.
El motor con IA debe generar propuestas de equipos en ≤ 20 segundos para conjuntos de hasta 100 perfiles simultáneos, considerando el tiempo de inferencia del modelo LLM/SML (Categoría: Rendimiento, Prioridad: Alta)

RNF2. Disponibilidad del sistema.
La plataforma debe estar disponible el 99% del tiempo durante el período académico, con ventanas de mantenimiento programadas fuera del horario lectivo (lunes–viernes, 6 a.m.–10 p.m.). (Categoría: Disponibilidad, Prioridad: Alta)

RNF3. Seguridad de credenciales.
Las contraseñas deben almacenarse con hashing bcrypt (mínimo 10 rondas). Los tokens JWT deben expirar en 24 horas y el sistema debe invalidar tokens revocados en ≤ 500 ms. (Categoría: Seguridad, Prioridad: Alta)

RNF4. Escalabilidad del esquema de grafo.
La base de datos Neo4j debe soportar hasta 1000 nodos de usuario y 50000 relaciones sin degradación de consultas superior al 20% respecto al tiempo base medido con 100 nodos. (Categoría: Escalabilidad, Prioridad: Media)

RNF5. Usabilidad de la interfaz.
Un usuario nuevo debe ser capaz de completar su perfil (habilidades + disponibilidad) en ≤ 5 minutos sin asistencia. El 90% de los pasos clave deben requerir ≤ 3 clics desde el menú principal. (Categoría: Usabilidad, Prioridad: Media)

RNF6. Seguridad de documentos cargados.

Los archivos adjuntos (pdf o txt) cargados por los usuarios solo deben ser procesados por el servicio LLM/SLM y no almacenados permanentemente en el sistema. El contenido extraído y normalizado es lo que se persiste en Neo4j. (Categoría: Privacidad, Prioridad: Alta)

ESTRUCTURA DEL SISTEMA.

Componentes del sistema.

1. Modulo de autenticación de usuario. Responsable del registro e inicio de sesión de usuarios. Gestiona la generación y validación de tokens JWT, así como el almacenamiento seguro de credenciales.

2. Modulo de gestión de perfiles. Encargado de la administración de cuentas de usuario y la exposición del perfil público. Incluye el procesamiento de habilidades expresadas en lenguaje natural o mediante documentos adjuntos (PDF/TXT): recibe el texto/archivo, lo envía al Servicio de Procesamiento IA, y persiste las habilidades normalizadas en Neo4j. También gestiona la disponibilidad horaria e información personal del usuario.

3. Servicio de procesamiento de habilidades, experiencia y disponibilidad (LLM/SLM). Microservicio encargado de procesar las entradas de habilidades, experiencia y disponibilidad en lenguaje natural o documentos cargados por los usuarios. Utiliza un modelo SLM/LLM (vía API externa o modelo ligero desplegado en contenedor) para extraer habilidades, normalizarlas a un vocabulario controlado, inferir niveles de dominio y años de experiencia cuando sea posible, identificar franjas de disponibilidad horaria expresadas en texto libre, y agrupar todo en categorías (técnicas, blandas, de dominio). Retorna un objeto estructurado que el módulo de gestión de perfiles persistirá en Neo4j.

4. Módulo de emparejamiento con IA. Componente central del sistema. En lugar de ejecutar exclusivamente consultas Cypher deterministicas, este motor utiliza un modelo de IA que recibe como contexto: los perfiles de candidatos (habilidades, disponibilidad, experiencia y el historial de retroalimentación), los requerimientos del proyecto (roles y habilidades necesarias) y las reglas de negocio definidas. El modelo evalúa las combinaciones posibles, aplica las reglas de negocio de forma inteligente (tolerando variaciones semánticas en habilidades) y retorna propuestas de equipos con puntajes de compatibilidad y justificaciones. Las consultas Cypher siguen siendo utilizadas para la recuperación de datos del grafo.

5. Módulo de gestión de proyectos. Permite a líderes crear y configurar proyectos, definiendo nombre, descripción, objetivos y los roles requeridos con sus habilidades asociadas. Sirve como entrada principal para que el motor de emparejamiento filtre y proponga los perfiles más adecuados. También soporta búsquedas manuales de candidatos mediante filtros por habilidades, experiencia y disponibilidad.

6. Módulo de gestión de equipos y tareas. Gestiona la composición del equipo una vez formado. Permite al líder asignar y modificar roles, crear tareas dirigidas a uno o más integrantes y administrar el ciclo de vida de cada tarea a través de los estados:

Pendiente, En Proceso, En Revisión y Completado. Soporta el flujo de revisión entre integrantes y el líder, incluyendo aprobación o rechazo de tareas finalizadas.

7. Módulo de retroalimentación e historial. Gestiona el proceso de evaluación mutua al cierre de cada proyecto. Permite a los integrantes calificar y comentar la participación de sus compañeros. Almacena las retroalimentaciones asociadas al perfil de cada usuario, organizadas por proyecto, y expone estos datos al motor de emparejamiento para enriquecer futuros procesos de formación de equipos.

8. Capa de persistencia (Neo4j). Responsable del almacenamiento y consulta de todos los datos del sistema mediante un modelo de grafos. Representa como nodos a los usuarios, habilidades, proyectos, equipos, experiencia y disponibilidad y como relaciones las conexiones entre ellos (pertenece a, tiene habilidad, trabaja en, evaluó a, entre otras). Provee las consultas Cypher que alimentan el motor de emparejamiento y los módulos de perfil e historial.

9. Capa de API REST (Backend). Desarrollada en Node.js + Express con TypeScript. Expone los endpoints consumidos por el frontend, aplica la lógica de negocio de cada módulo, gestiona la autenticación mediante JWT y orquesta la comunicación entre los demás componentes, el servicio LLM/SLM y la base de datos.

10. Interfaz de usuario (Frontend). Desarrollada en React + Vite con TypeScript. Presenta las vistas de registro, perfil, gestión de proyectos, tablero de tareas y retroalimentación. Consume la API REST del backend y utiliza los prototipos de Figma como referencia visual vinculante para su implementación.

Relación entre componentes.

1. Interacción Frontend-Backend: La Interfaz de Usuario (React) realiza peticiones mediante API REST a la lógica de negocio en Node.js. Todas las peticiones protegidas incluyen un Token JWT generado por el Módulo de Autenticación.

2. Flujo de procesamiento de habilidades, experiencia y disponibilidad: Cuando el usuario ingresa texto libre o carga un documento (PDF/TXT), el backend envía el contenido al Servicio de Procesamiento LLM/SLM. El servicio retorna las habilidades, experiencia y disponibilidad normalizadas y agrupadas, que el Módulo de Gestión de Perfiles persiste en Neo4j como nodos y relaciones.

3. Flujo del motor de IA: Cuando el Módulo de Proyectos recibe nuevos requerimientos, el Motor de Emparejamiento con IA consulta la Capa de Persistencia (Neo4j) para recuperar los perfiles de candidatos relevantes mediante consultas Cypher, y luego envía el contexto completo (perfiles + requerimientos + reglas de negocio) al modelo de IA para obtener las propuestas de equipos óptimas.

4. Ciclo de Tareas y Retroalimentación: El módulo de Equipos y Tareas actualiza el estado de los nodos en la base de datos. Al finalizar, el Módulo de

Retroalimentación escribe nuevas relaciones de evaluación en el grafo, las cuales el Motor de Emparejamiento utilizará como contexto adicional en futuros procesos.

Diseño del modelo de datos.

Nodos (Entidades Principales)

- **Usuario:** Contiene nombre, email y password_hash. (Se elimina el atributo disponibilidad porque ahora es un nodo).
- **Habilidad:** Contiene el nombre de la competencia (normalizado por el LLM) y un booleano es_tecnica.
- **Proyecto:** Contiene nombre, descripcion y estado.
- **Tarea:** Contiene titulo, descripcion y estado_actual.
- **Experiencia:** Contiene titulo_cargo, empresa, duracion_meses y resumen_ia (extraído del CV).
- **Disponibilidad:** Contiene el tipo_disponibilidad (ej. "Tiempo Completo", "Remoto", "Mañanas").

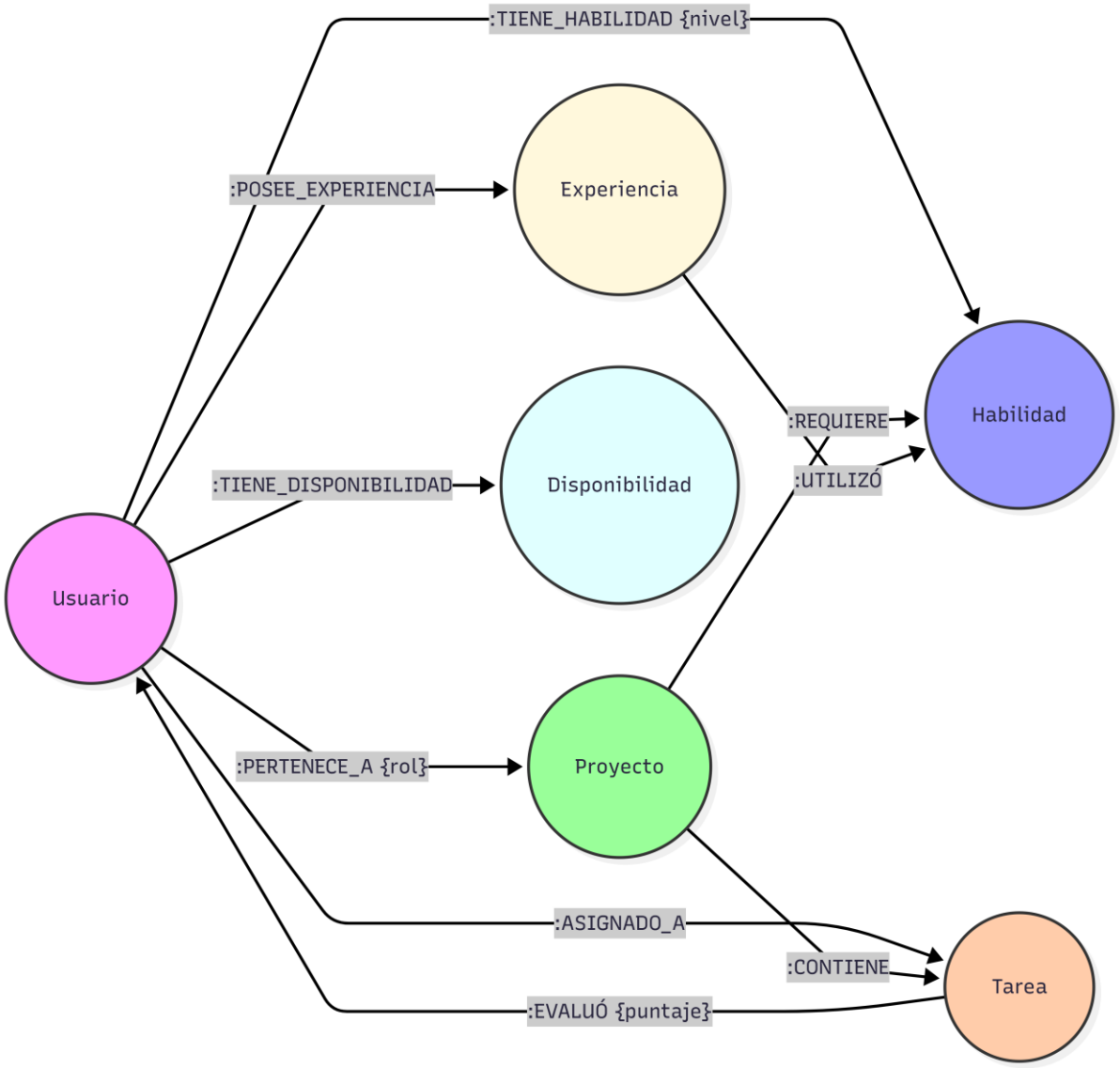
Relaciones (Conexiones)

Esta tabla refleja exactamente lo que muestra tu código Mermaid, incluyendo las jerarquías y las nuevas entidades para el **RF9**.

Relación (Cypher Syntax)	Descripción Funcional	Atributos Clave
(:Usuario)-[:TIENE_HABILIDAD]->(:Habilidad)	Define el dominio técnico y el nivel de experticia del usuario.	nivel (Básico, Intermedio, Avanzado)
(:Usuario)-[:PERTENECE_A]->(:Proyecto)	Vincula a los usuarios con sus proyectos activos y su jerarquía.	rol (Líder, Miembro)
(:Usuario)-[:ASIGNADO_A]->(:Tarea)	Establece quién es el responsable directo de ejecutar una tarea.	fecha_asignacion, estado
(:Usuario)-[:POSEE_EXPERIENCIA]->(:Experiencia)	Conecta al usuario con su historial laboral detallado.	fecha_inicio, es_actual

(:Usuario)-[:TIENE_DISPONIBILIDAD]->(:Disponibilidad)	**Permite filtrar usuarios según su tiempo o modalidad disponible.**	prioridad_usuario
(:Proyecto)-[:REQUIERE]->(:Habilidad)	Especifica las competencias necesarias para el éxito del proyecto.	prioridad, años_experiencia
(:Proyecto)-[:CONTIENE]->(:Tarea)	Desglosa el proyecto en actividades accionables del tablero.	orden_logico
(:Tarea)-[:EVALUO]->(:Usuario)	El resultado de la tarea genera una calificación sobre el desempeño.	puntaje (1-5), comentario
(:Experiencia)-[:UTILIZO]->(:Habilidad)	Cruza la experiencia laboral con las habilidades aplicadas.	relevancia

Diagrama de base de datos.



DIVISIÓN POR MÓDULOS.

TeamMatch se divide en ocho módulos funcionales. Cada módulo tiene responsabilidades bien delimitadas para facilitar el desarrollo paralelo en equipo y la trazabilidad hacia los requerimientos funcionales.

ID	Módulo	Descripción	Responsable	RFs asociados

M1	**Autenticación de usuario**	Registro, inicio de sesión, generación y validación de tokens JWT, almacenamiento de credenciales.	Backend Engineering	RF1, RF2
M2	**Gestión de perfiles**	Gestión de cuentas, perfil público, habilidades con niveles de dominio, disponibilidad horaria e información personal.	Backend Engineering	RF3, RF4, RF5
M3	**Motor de emparejamiento**	Ejecuta reglas de negocio sobre el grafo Neo4j para calcular puntajes de compatibilidad y generar propuestas de equipos equilibrados. Consume historial de retroalimentación.	Backend Engineering	RF6, RF7
M4	**Gestión de proyectos**	Creación y configuración de proyectos con roles y habilidades requeridas. Búsqueda manual de candidatos por filtros de habilidades, experiencia y disponibilidad.	Backend Engineering	RF8, RF9
M5	**Gestión de equipos y tareas**	Composición del equipo, asignación y modificación de roles, ciclo de vida de tareas (Pendiente → En Proceso → En Revisión → Completado), aprobación o rechazo por el líder.	Backend Engineering	RF10, RF11, RF12, RF13
M6	**Retroalimentación e historial**	Evaluación mutua al cierre de proyectos, almacenamiento de retroalimentaciones por perfil y proyecto, exposición al motor de emparejamiento.	Backend Engineering	RF14, RF15
M7	**Capa de persistencia (Neo4j)**	Almacenamiento y consulta de todos los datos mediante modelo de grafos. Nodos: usuarios, habilidades, proyectos, equipos. Relaciones: pertenece_a,	DBA	Todos

		tiene_habilidad, trabaja_en, evaluó_a.		
M8	**API REST (Backend)**	Node.js + Express + TypeScript. Expone endpoints del frontend, aplica lógica de negocio, gestiona autenticación JWT y orquesta la comunicación entre módulos y base de datos.	Backend Engineering	Todos
M9	**Interfaz de usuario (Frontend)**	React + Vite + TypeScript. Vistas: registro, perfil, gestión de proyectos, tablero de tareas y retroalimentación. Consume la API REST. Usa prototipos Figma como referencia visual.	Frontend Engineering	Todos

Flujo de registro de habilidades, experiencia y disponibilidad via LLM/SLM

El siguiente flujo describe el proceso end-to-end para el registro de habilidades, experiencia y disponibilidad mediante texto libre o documentos:

1. El usuario accede a la sección de perfil y elige entre: (a) ingresar texto libre describiendo sus competencias, trayectoria profesional y disponibilidad horaria, o (b) cargar un archivo PDF o TXT (por ejemplo, un CV o una descripción personal).
2. El frontend envía el texto o el archivo (como base64 o multipart) al endpoint correspondiente del backend.
3. El backend reenvía el contenido al Servicio de Procesamiento de Habilidades, Experiencia y Disponibilidad (LLM/SLM).
4. El modelo SLM/LLM analiza el texto/documento e identifica y extrae tres categorías de información: (a) **habilidades** técnicas, blandas y de dominio, normalizando sus nombres (e.g., 'React' → 'React.js', 'liderar equipos' → 'Liderazgo de equipos') e infiriendo niveles de dominio cuando hay indicadores explícitos o contextuales; (b) **experiencia**, extrayendo años de experiencia general o por habilidad, roles desempeñados y contextos relevantes; y (c) **disponibilidad**, identificando franjas horarias, días disponibles y dedicación semanal expresados en lenguaje natural (e.g., 'disponible tardes entre semana y fines de semana' → días: lunes-viernes, franja: tarde; sábado-domingo, franja: completa). El modelo retorna un JSON estructurado con los tres bloques de información.
5. El backend presenta al usuario la información extraída — habilidades, experiencia y disponibilidad — para su revisión, corrección y confirmación.

6. Una vez confirmada, el backend persiste la información en Neo4j: las habilidades como nodos Habilidad con relaciones TIENE_HABILIDAD hacia el nodo Usuario (incluyendo nivel de dominio y el atributo fuente: texto_libre, documento_pdf o documento_txt); la experiencia como atributos enriquecidos en la relación TIENE_HABILIDAD o en el nodo Usuario según corresponda; y la disponibilidad como estructura de franjas horarias en el nodo Usuario.

Flujo del motor de emparejamiento con IA

El siguiente flujo describe cómo el motor de IA aplica las reglas de negocio para generar propuestas de equipos:

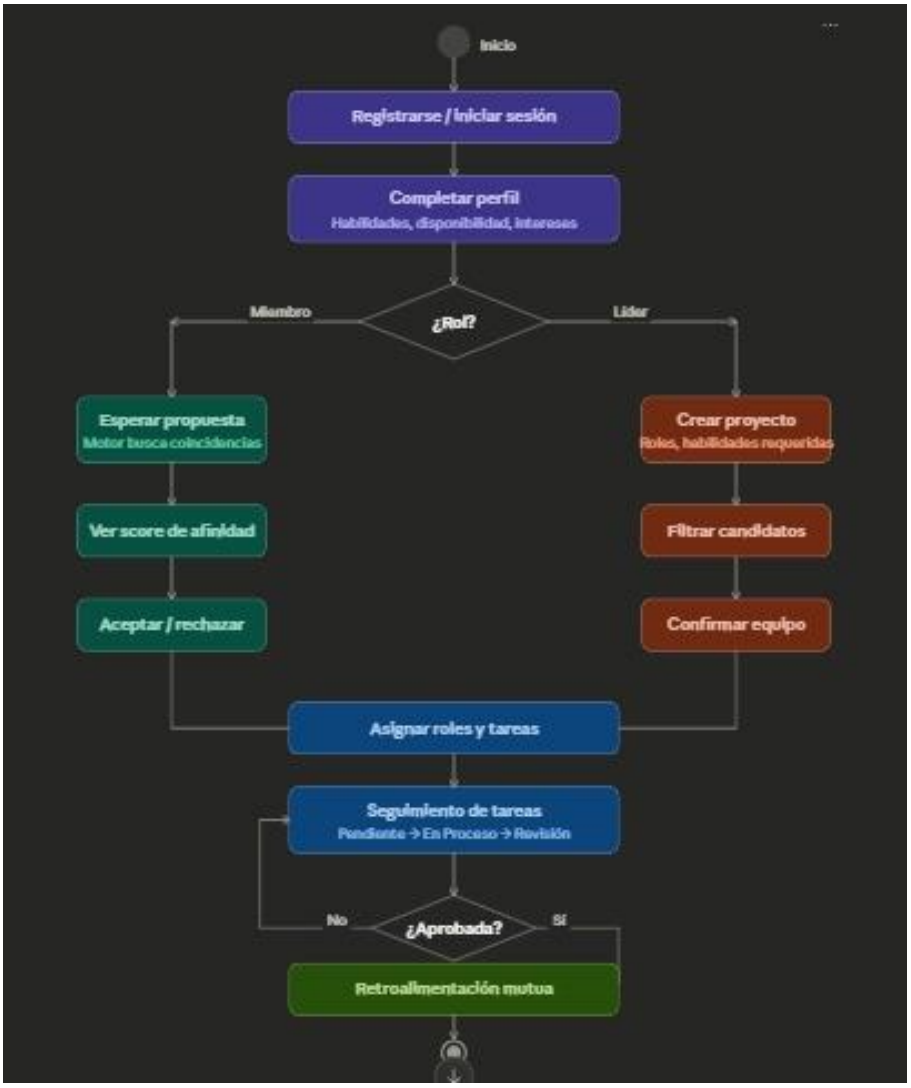
1. El líder crea un proyecto y define los roles necesarios con sus habilidades requeridas, nivel de experiencia esperado y franjas horarias de trabajo del proyecto.
2. El motor de emparejamiento ejecuta consultas Cypher en Neo4j para recuperar los candidatos cuyos perfiles tienen alguna coincidencia con las habilidades requeridas, considerando únicamente aquellos cuyas habilidades fueron normalizadas y persistidas previamente por el Servicio de Procesamiento LLM/SLM.
3. Se construye un contexto estructurado que incluye: lista de candidatos con sus habilidades normalizadas, niveles de dominio, años de experiencia por habilidad, disponibilidad horaria estructurada (días y franjas extraídas por el LLM/SLM) y retroalimentaciones previas de proyectos anteriores; requerimientos del proyecto (roles, habilidades, nivel de experiencia y disponibilidad requerida); y reglas de negocio (complementariedad de habilidades, balance de niveles de dominio, alineación de experiencia con el rol, compatibilidad de disponibilidad horaria e historial de colaboración entre candidatos).
4. El contexto es enviado al modelo de IA (LLM/SLM), que evalúa las combinaciones posibles aplicando las reglas de negocio de forma inteligente: cruza la disponibilidad estructurada de cada candidato contra las franjas del proyecto, pondera la experiencia acumulada frente al nivel requerido por rol, y tolera variaciones semánticas residuales entre habilidades, considerando también factores cualitativos derivados del historial de retroalimentación.
5. El modelo retorna las propuestas de equipo con sus puntajes de afinidad desglosados por criterio (habilidades, experiencia, disponibilidad e historial) y una justificación legible por el usuario que explica por qué cada candidato fue seleccionado para su rol.
6. El frontend presenta las propuestas al líder, quien puede aceptarlas, modificarlas o solicitar nuevas propuestas.

Primer Diagrama de Actividad

El diagrama de actividad describe el flujo completo de la plataforma desde el punto de vista del usuario, incluyendo la bifurcación de roles (Miembro vs. Líder) y la convergencia en la gestión compartida del equipo.

Actividades principales del flujo:

- **Inicio**: El usuario accede a la plataforma y se registra o inicia sesión.
- **Perfil**: Completa habilidades, intereses y disponibilidad horaria.
- **Bifurcación de rol**: El sistema distingue entre Miembro (espera propuesta del motor) y Líder (crea proyecto, filtra candidatos y confirma equipo).
- **Convergencia**: Ambos roles confluyen en la gestión compartida del equipo y el tablero de tareas.
- **Seguimiento**: Las tareas pasan por los estados Pendiente → En Proceso → En Revisión → Completado. Si el líder rechaza, vuelve a En Proceso.
- **Cierre**: Al finalizar el proyecto, todos los miembros realizan retroalimentación mutua que enriquece el historial de perfiles.



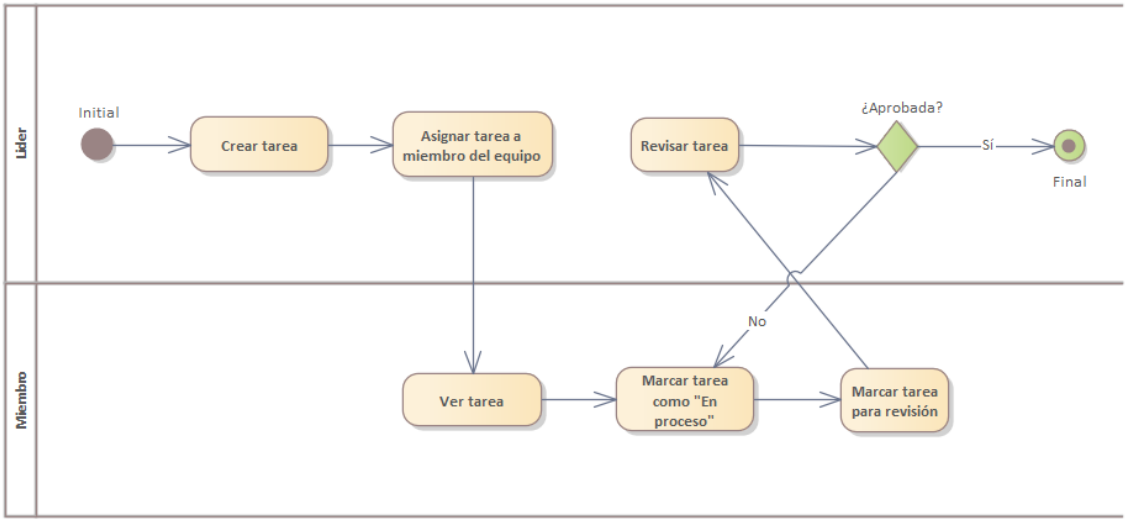
Segundo Diagrama de Actividad: Gestión de tareas

El siguiente diagrama de actividades representa el flujo del proceso de gestión de tareas dentro del sistema, diferenciando las responsabilidades entre dos roles: Líder y Miembro del equipo.

El proceso inicia cuando el Líder crea una nueva tarea. Posteriormente, dicha tarea es asignada a un miembro del equipo específico. Una vez asignada, el Miembro puede visualizar la tarea y marcarla como “En proceso”, indicando que ha comenzado a trabajar en ella.

Cuando el miembro considera que la tarea está lista, la marca como “para revisión”, lo que activa nuevamente la intervención del Líder. A continuación, el líder revisa la tarea y toma una decisión:

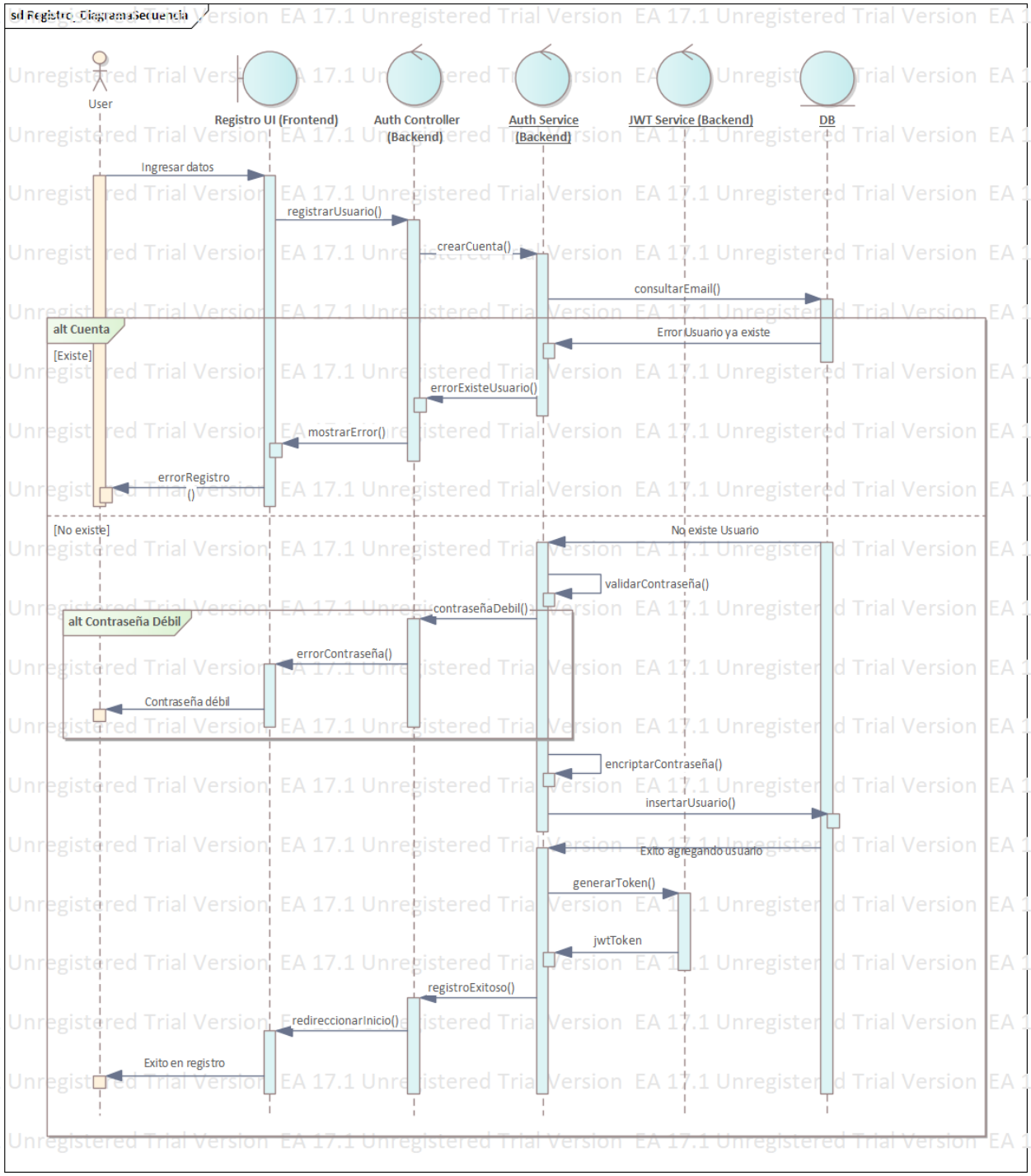
- Si la tarea es aprobada, el proceso finaliza.
- Si la tarea no es aprobada, se devuelve al flujo del miembro, quien debe continuar trabajando en ella (retornando al estado “En proceso”) y posteriormente reenviarla para revisión.



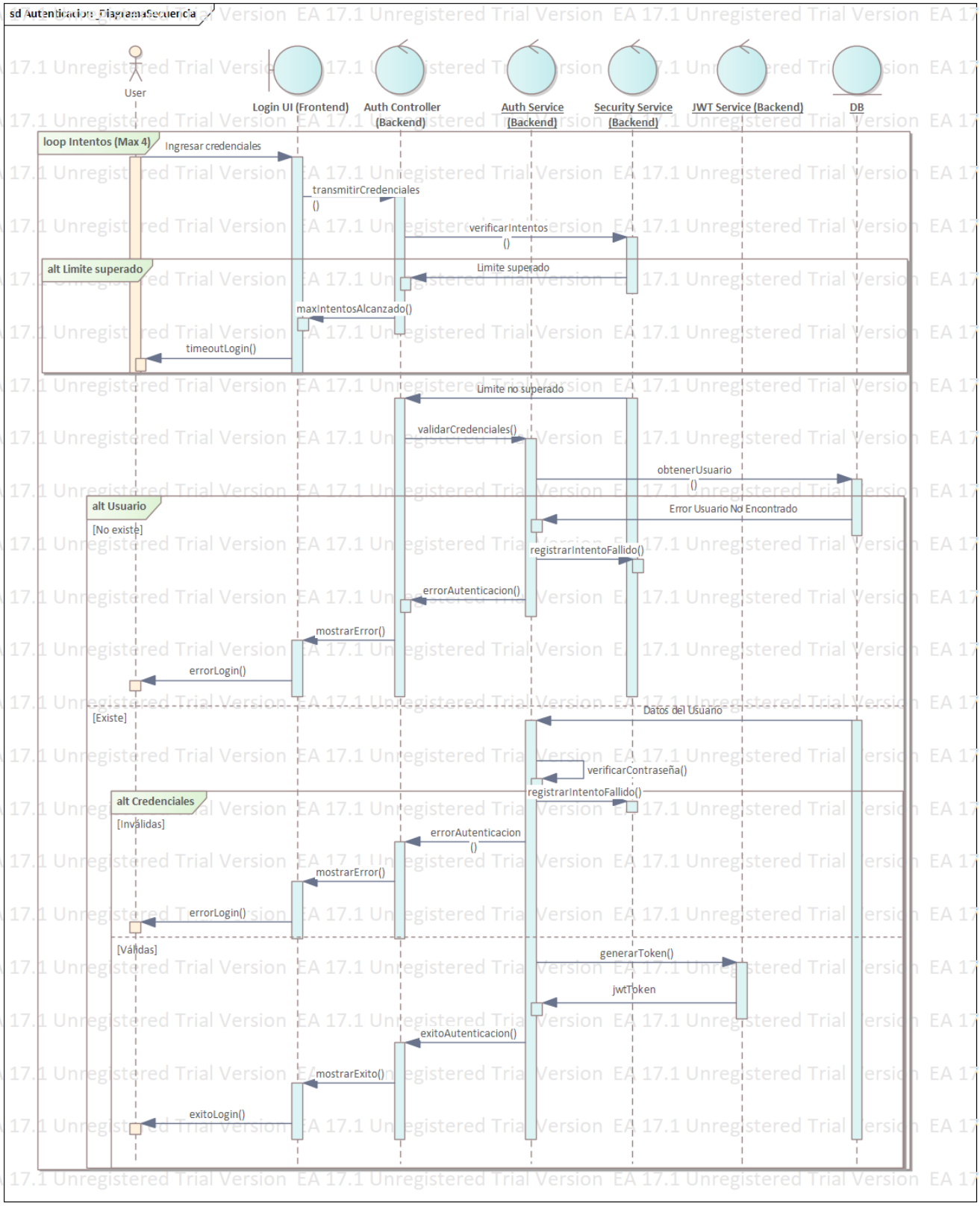
ARTEFACTOS OBLIGATORIOS.

Diagramas de Secuencia.

Registro

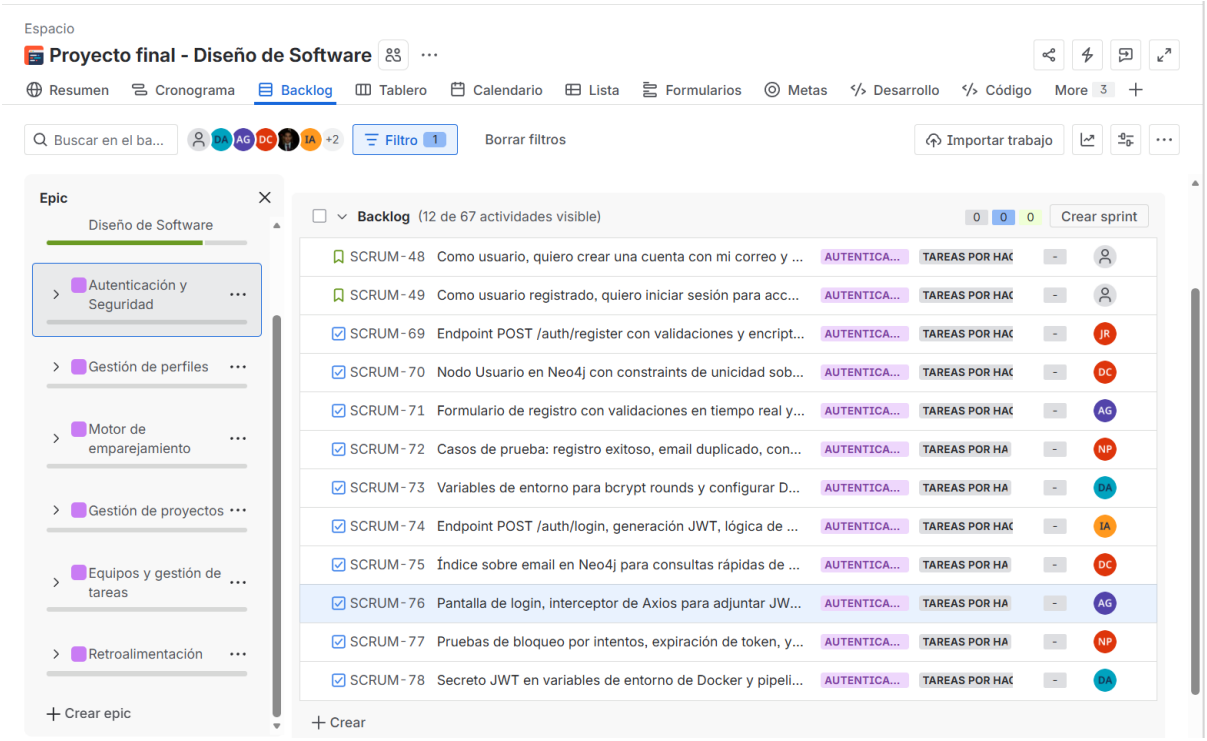


Autenticación



Consideraciones del diagrama de secuencia de autenticación: Se tienen en cuenta aspectos como evitar el reinicio del número de intentos de acceso realizados al momento de reiniciar el sistema para futuras variaciones del diagrama. No se representa como un diagrama final.

Historias de usuarios en Jira.



Espacio

Proyecto final - Diseño de Software

Resumen | Cronograma | Backlog | Tablero | Calendario | Lista | Formularios | Metas | Desarrollo | Código | More 3

Buscar en el ba... | DA AG DC IA +2 | Filtro 1 | Borrar filtros | Importar trabajo

Epic

Diseño de Software

- Autenticación y Seguridad
- Gestión de perfiles
- Motor de emparejamiento
- Gestión de proyectos
- Equipos y gestión de tareas
- Retroalimentación

SCRUM-56	Como usuario quiero definir mi disponibilidad horaria se...	GESTIÓN DE...	TAREAS POR HAC	-	
SCRUM-57	Como usuario quiero ver el perfil público de otro usuari...	GESTIÓN DE...	TAREAS POR HAC	-	
SCRUM-79	Endpoints CRUD /profile/skills para crear, editar y elimin...	GESTIÓN DE...	TAREAS POR HAC	-	
SCRUM-80	Nodo Habilidad y relación :TIENE_HABILIDAD {nivel} c...	GESTIÓN DE...	TAREAS POR HAC	-	DC
SCRUM-81	Componente de gestión de habilidades con etiquetas e...	GESTIÓN DE...	TAREAS POR HAC	-	AG
SCRUM-82	Pruebas de CRUD de habilidades, validación de niveles ...	GESTIÓN DE...	TAREAS POR HAC	-	NP
SCRUM-83	Endpoint PUT /profile/availability con estructura de días...	GESTIÓN DE...	TAREAS POR HAC	-	
SCRUM-84	Propiedad disponibilidad en el nodo Usuario como array...	GESTIÓN DE...	TAREAS POR HAC	-	DC
SCRUM-85	Grilla semanal interactiva para seleccionar disponibilida...	GESTIÓN DE...	TAREAS POR HAC	-	AG
SCRUM-86	Endpoint GET /users/:id/profile con proyección que excl...	GESTIÓN DE...	TAREAS POR HAC	-	
SCRUM-87	Verificar que datos sensibles no se expongan en respue...	GESTIÓN DE...	TAREAS POR HAC	-	NP
SCRUM-88	Vista de perfil público con habilidades, retroalimentacio...	GESTIÓN DE...	TAREAS POR HAC	-	AG

+ Crear

Espacio

Proyecto final - Diseño de Software

Resumen | Cronograma | Backlog | Tablero | Calendario | Lista | Formularios | Metas | Desarrollo | Código | More 3

Buscar en el ba... | DA AG DC IA +2 | Filtro 1 | Borrar filtros | Importar trabajo

Epic

Diseño de Software

- Autenticación y Seguridad
- Gestión de perfiles
- Motor de emparejamiento
- Gestión de proyectos
- Equipos y gestión de tareas
- Retroalimentación

Backlog (11 de 67 actividades visible) | 0 | 0 | 0 | Crear sprint

SCRUM-58	Como líder quiero que el sistema genere propuestas de ...	MOTOR DE E...	TAREAS POR HAC	-	
SCRUM-59	Como usuario quiero ver el score de afinidad de cada p...	MOTOR DE E...	TAREAS POR HAC	-	
SCRUM-89	Motor de matchmaking: query Cypher que calcula cami...	MOTOR DE E...	TAREAS POR HAC	-	
SCRUM-90	Query Cypher optimizada para el matching con índices ...	MOTOR DE E...	TAREAS POR HAC	-	DC
SCRUM-91	Vista de propuestas de equipo con tarjetas de miembro,...	MOTOR DE E...	TAREAS POR HAC	-	AG
SCRUM-92	Pruebas de rendimiento (≤3s), pruebas de calidad de pr...	MOTOR DE E...	TAREAS POR HAC	-	NP
SCRUM-93	Configurar Neo4j con memoria y caché optimizados en ...	MOTOR DE E...	TAREAS POR HAC	-	DA
SCRUM-94	Endpoint que retorna score de compatibilidad con desgl...	MOTOR DE E...	TAREAS POR HAC	-	JR
SCRUM-95	Incorporar peso de relaciones :EVALUÓ {puntaje} al cál...	MOTOR DE E...	TAREAS POR HAC	-	DC
SCRUM-96	Componente de score con barra de progreso, desglose ...	MOTOR DE E...	TAREAS POR HAC	-	AG
SCRUM-97	Verificar consistencia del score con los datos de perfile...	MOTOR DE E...	TAREAS POR HAC	-	NP

+ Crear

Espacio

Proyecto final - Diseño de Software

Resumen | Cronograma | Backlog | Tablero | Calendario | Lista | Formularios | Metas | Desarrollo | Código | More 3

Buscar en el ba... | DA AG DC IA +2 | Filtro 1 | Borrar filtros | Importar trabajo

Epic

- Diseño de Software
- Autenticación y Seguridad
- Gestión de perfiles
- Motor de emparejamiento
- Gestión de proyectos
- Equipos y gestión de tareas
- Retroalimentación

No hay nada que coincida con este filtro

+ Crear

Backlog (8 de 67 actividades visible) | 0 0 0 | Crear sprint

SCRUM-66	Como líder quiero crear un proyecto con roles y habilida...	GESTIÓN DE...	TAREAS POR HAC	-	
SCRUM-67	Como líder quiero buscar candidatos manualmente con ...	GESTIÓN DE...	TAREAS POR HAC	-	
SCRUM-98	Endpoint POST /projects con validaciones y creación de...	GESTIÓN DE...	TAREAS POR HAC	-	DA
SCRUM-99	Nodo Proyecto con relación :REQUIERE {prioridad, año...	GESTIÓN DE...	TAREAS POR HAC	-	DC
SCRUM-100	Formulario de creación de proyecto con sección diná...	GESTIÓN DE...	TAREAS POR HAC	-	AG
SCRUM-101	Endpoint GET /users/search con parámetros de filtro, ...	GESTIÓN DE...	TAREAS POR HAC	-	IA
SCRUM-102	Panel de búsqueda con filtros combinables, resultado...	GESTIÓN DE...	TAREAS POR HAC	-	AG
SCRUM-103	Probar combinaciones de filtros, resultados vacíos, y ...	GESTIÓN DE...	TAREAS POR HAC	-	NP

+ Crear

Espacio

Proyecto final - Diseño de Software

Resumen | Cronograma | Backlog | Tablero | Calendario | Lista | Formularios | Metas | Desarrollo | Código | More 3

Buscar en el ba... | DA AG DC IA +2 | Filtro 1 | Borrar filtros | Importar trabajo

Epic

- Diseño de Software
- Autenticación y Seguridad
- Gestión de perfiles
- Motor de emparejamiento
- Gestión de proyectos
- Equipos y gestión de tareas
- Retroalimentación

Backlog (11 de 67 actividades visible) | 0 0 0 | Crear sprint

SCRUM-63	Como líder quiero asignar tareas a miembros del equipo...	EQUIPOS Y ...	TAREAS POR HAC	-	
SCRUM-64	Como líder quiero modificar la composición del equipo ...	EQUIPOS Y ...	TAREAS POR HAC	-	
SCRUM-104	Endpoints CRUD /tasks con máquina de estados (Pen...	EQUIPOS Y ...	TAREAS POR HAC	-	
SCRUM-105	Nodo Tarea con relaciones :CONTIENE (Proyecto) y :A...	EQUIPOS Y ...	TAREAS POR HAC	-	DC
SCRUM-106	Tablero Kanban con columnas de estados, drag & dro...	EQUIPOS Y ...	TAREAS POR HAC	-	AG
SCRUM-107	Pruebas de flujo completo del ciclo de tarea, transicio...	EQUIPOS Y ...	TAREAS POR HAC	-	NP
SCRUM-108	Configurar WebSockets o polling en el stack Docker p...	EQUIPOS Y ...	TAREAS POR HAC	-	DA
SCRUM-109	Endpoints PUT /teams/:id/members para agregar/rem...	EQUIPOS Y ...	TAREAS POR HAC	-	JR
SCRUM-110	Query Cypher para actualizar relaciones :PERTENECE...	EQUIPOS Y ...	TAREAS POR HAC	-	DC
SCRUM-111	Modal de gestión de equipo con visualización actual, a...	EQUIPOS Y ...	TAREAS POR HAC	-	AG
SCRUM-112	Probar remoción de miembro con tareas activas, ...	EQUIPOS Y ...	TAREAS POR HA	-	NP

+ Crear

Espacio

Proyecto final - Diseño de Software

Resumen | Cronograma | Backlog | Tablero | Calendario | Lista | Formularios | Metas | Desarrollo | Código | More 3

Buscar en el ba... | DA AG DC IA +2 | Filtro 1 | Borrar filtros | Importar trabajo

Epic

Diseño de Software

- Autenticación y Seguridad
- Gestión de perfiles
- Motor de emparejamiento
- Gestión de proyectos
- Equipos y gestión de tareas
- Retroalimentación

Sprint 1 2 abr – 9 abr (0 de 34 actividades visible) | 0 0 0 | Completar sprint

No hay nada que coincida con este filtro

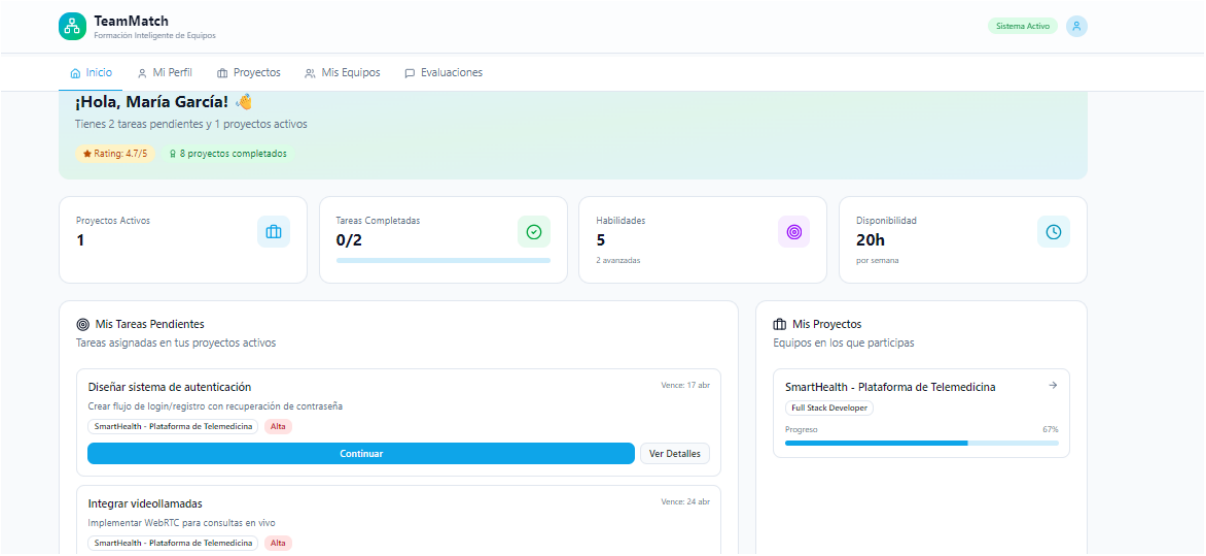
+ Crear

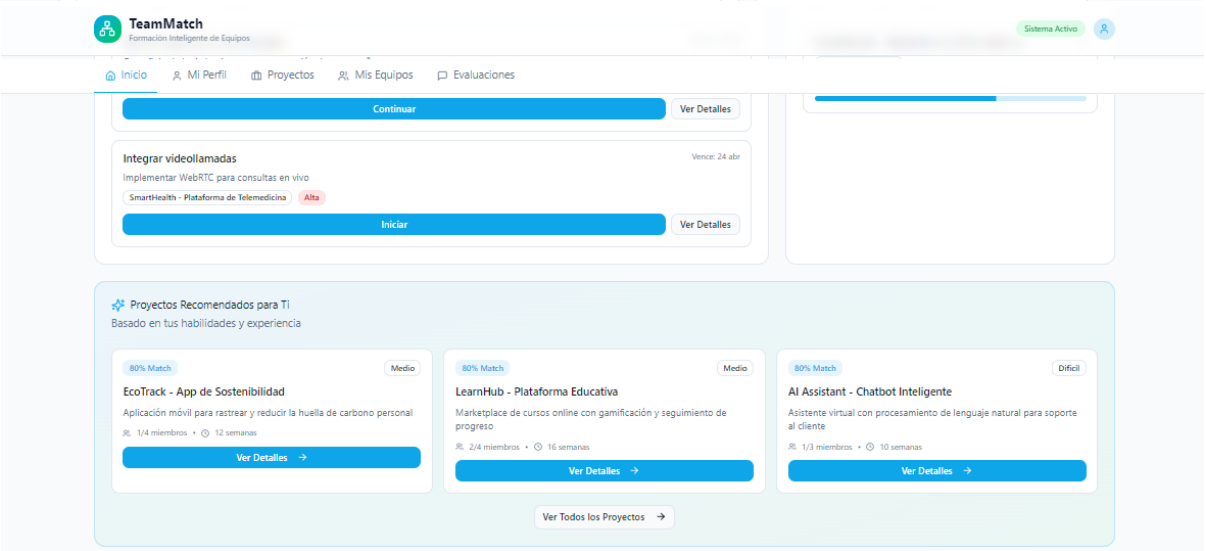
Backlog (6 de 67 actividades visible) | 0 0 0 | Crear sprint

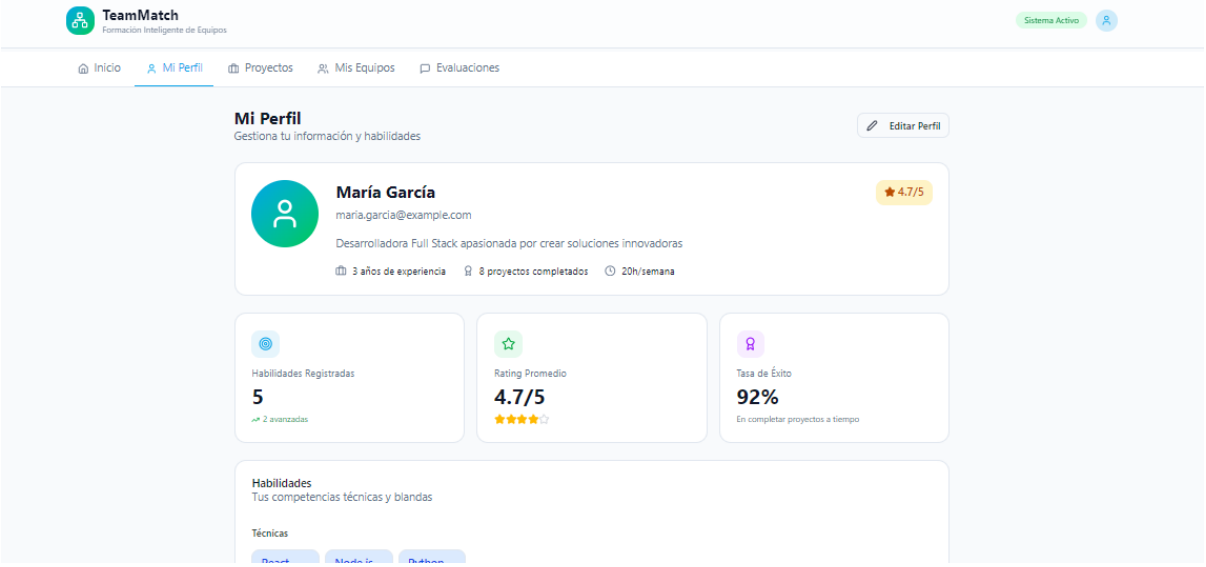
SCRUM-114	Como integrante quiero evaluar a mis compañeros al cie...	RETROALIM...	TAREAS POR HAC	-	
SCRUM-115	Endpoint POST /feedback al cerrar proyecto, con validac...	RETROALIM...	TAREAS POR HAC	-	
SCRUM-116	Relación :EVALUÓ {puntaje, comentario, proyecto_id} en...	RETROALIM...	TAREAS POR HAC	-	DC
SCRUM-117	Formulario de retroalimentación por compañero, vista de...	RETROALIM...	TAREAS POR HAC	-	AG
SCRUM-118	Verificar que solo se activa al cerrar proyecto, que el mo...	RETROALIM...	TAREAS POR HAC	-	NP
SCRUM-119	Backup programado de Neo4j en Docker Compose para ...	RETROALIM...	TAREAS POR HAC	-	DA

+ Crear

Prototipos en Figma.







TeamMatch
Formación Inteligente de Equipos

Sistema Activo

Inicio | Mi Perfil | Proyectos | Mis Equipos | Evaluaciones

Habilidades
Tus competencias técnicas y blandas

Técnicas

React | Node.js | Python

Blandas

Liderazgo | Comunicación

Intereses
Áreas y temas que te apasionan

Web Development | IA | UX Design | Startups

Disponibilidad
Tu tiempo disponible para proyectos

Horas por semana
20h

Horarios preferidos
Tardes | Fines de semana

TeamMatch
Formación Inteligente de Equipos

Sistema Activo

Inicio | Mi Perfil | Proyectos | Mis Equipos | Evaluaciones

Explorar Proyectos
Encuentra el equipo perfecto para ti

Crear Proyecto

Buscar proyectos...

Filtros

Categoría: Todas las categorías
Dificultad: Todas
Estado: Todos

Todos (4) | Disponibles (3) | En Progreso (1)

Abierto
EcoTrack - App de Sostenibilidad
Aplicación móvil para rastrear y reducir la huella de carbono personal
1/4 miembros | Medio
12 semanas | Mobile App
3 lugares disponibles
Habilidades requeridas: React Native | UI Design | API Development

En Progreso
SmartHealth - Plataforma de Telemedicina
Sistema de consultas médicas en línea con historial clínico digital
3/5 miembros | Difícil
20 semanas | Web Platform
Habilidades requeridas: Java | Security | React
Ver Detalles →

Abierto | 67% Match
LearnHub - Plataforma Educativa
Marketplace de cursos online con gamificación y seguimiento de progreso
2/4 miembros | Medio
16 semanas | Education
2 lugares disponibles
Habilidades requeridas: React | Node.js | Figma

TeamMatch
Formación Inteligente de Equipos

Sistema Activo

Inicio | Mi Perfil | Proyectos | Mis Equipos | Evaluaciones

1/4 miembros | Medio
12 semanas | Mobile App
3 lugares disponibles
Habilidades requeridas: React Native | UI Design | API Development
Ver Detalles →

3/5 miembros | Difícil
20 semanas | Web Platform
Habilidades requeridas: Java | Security | React
Ver Detalles →

2/4 miembros | Medio
16 semanas | Education
2 lugares disponibles
Habilidades requeridas: React | Node.js | Figma
Ver Detalles →

Abierto
AI Assistant - Chatbot Inteligente
Asistente virtual con procesamiento de lenguaje natural para soporte al cliente
1/3 miembros | Difícil
10 semanas | AI/ML
2 lugares disponibles
Habilidades requeridas: Python | Machine Learning | NLP
Ver Detalles →

TeamMatch
Formación Inteligente de Equipos

Sistema Activo

Inicio | Mi Perfil | Proyectos | Mis Equipos | Evaluaciones

Buscar proyectos...

Filtros

Categoría: Todas las categorías

Dificultad: Todas

Estado: Todos

Todos (4) | Disponibles (3) | En Progreso (1)

Abierto

EcoTrack - App de Sostenibilidad
Aplicación móvil para rastrear y reducir la huella de carbono personal

1/4 miembros | Medio

12 semanas | Mobile App

3 lugares disponibles

Habilidades requeridas:
React Native | UI Design | API Development

Ver Detalles →

Abierto | 67% Match

LearnHub - Plataforma Educativa
Marketplace de cursos online con gamificación y seguimiento de progreso

2/4 miembros | Medio

16 semanas | Education

2 lugares disponibles

Habilidades requeridas:
React | Node.js | Figma

Ver Detalles →

Abierto

AI Assistant - Chatbot Inteligente
Asistente virtual con procesamiento de lenguaje natural para soporte al cliente

1/3 miembros | Difícil

10 semanas | AI/ML

2 lugares disponibles

Habilidades requeridas:
Python | Machine Learning | NLP

Ver Detalles →

TeamMatch
Formación Inteligente de Equipos

Sistema Activo

Inicio | Mi Perfil | Proyectos | Mis Equipos | Evaluaciones

Explorar Proyectos
Encuentra el equipo perfecto para ti

Crear Proyecto

Buscar proyectos...

Filtros

Categoría: Todas las categorías

Dificultad: Todas

Estado: Todos

Todos (4) | Disponibles (3) | En Progreso (1)

En Progreso

SmartHealth - Plataforma de Telemedicina
Sistema de consultas médicas en línea con historial clínico digital

3/5 miembros | Difícil

20 semanas | Web Platform

Habilidades requeridas:
Java | Security | React

Ver Detalles →

TeamMatch
Formación Inteligente de Equipos

Sistema Activo

Inicio | Mi Perfil | Proyectos | Mis Equipos | Evaluaciones

Mis Equipos
Proyectos en los que colaboras

Proyectos Activos	Proyectos Completados	Tareas Completadas
1	0	8

Proyectos Activos
Equipos en los que estás trabajando actualmente

SmartHealth - Plataforma de Telemedicina | Full Stack Developer

Ver Proyecto →

Sistema de consultas médicas en línea con historial clínico digital

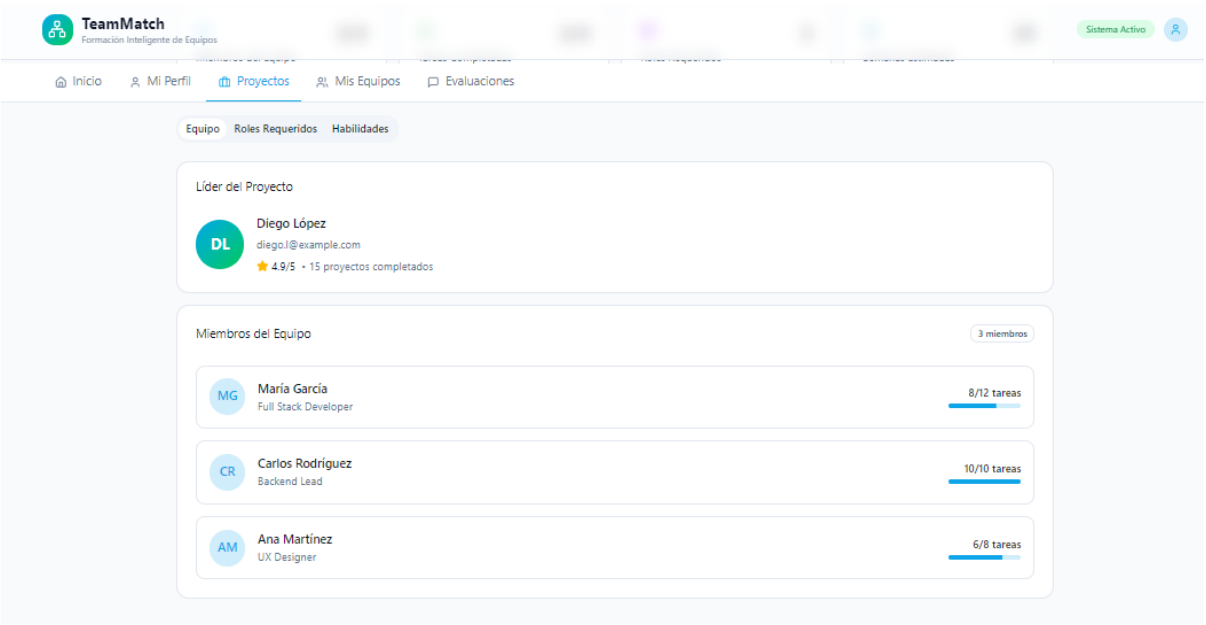
3 miembros | 20 semanas | Web Platform

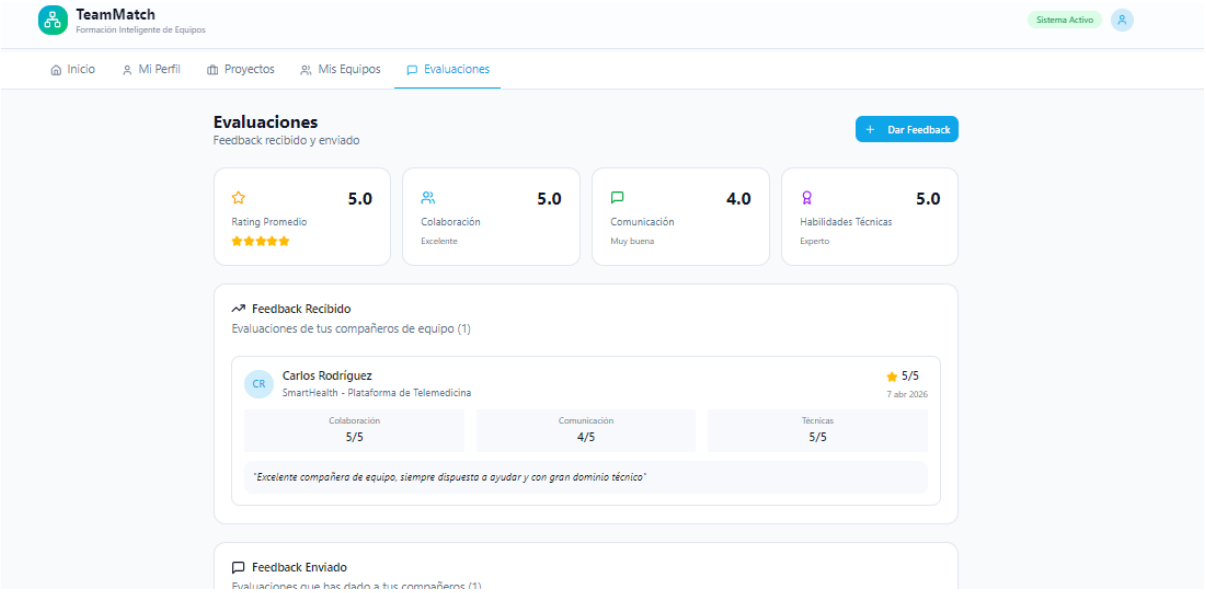
Mi Progreso
67% 8/12

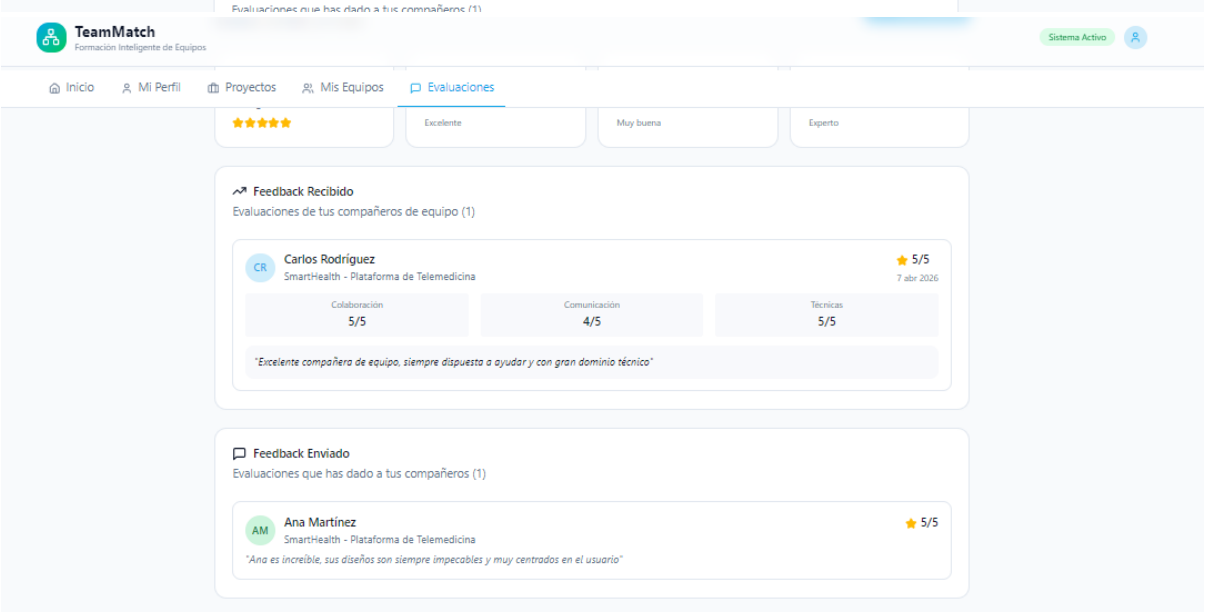
Tareas Pendientes
2 Normal
2 de alta prioridad

Equipo
MG CR AM
3 colaboradores

TeamMatch
Formacion Inteligente de Equipos
Sistema Activo
Inicio
Mi Perfil
Proyectos
Mis Equipos
Evaluaciones
Volver
SmartHealth - Plataforma de Telemedicina
Difícil
Sistema de consultas médicas en línea con historial clínico digital
Inicio: 28 de febrero de 2026
20 semanas
Web Platform
3/5
Miembros del Equipo
2/4
Tareas Completadas
3
Roles Requeridos
20
Semanas Estimadas
Equipo
Roles Requeridos
Habilidades
Líder del Proyecto
DL
Diego López
diego.l@example.com
4.9/5 • 15 proyectos completados
Miembros del Equipo
3 miembros


TeamMatch
Formación Inteligente de Equipos
Sistema Activo
Inicio
Mi Perfil
Proyectos
Mis Equipos
Evaluaciones
Equipo
Roles Requeridos
Habilidades
Líder del Proyecto
DL
Diego López
diego.l@example.com
4.9/5 • 15 proyectos completados
Miembros del Equipo
3 miembros
MG
María García
Full Stack Developer
8/12 tareas
CR
Carlos Rodríguez
Backend Lead
10/10 tareas
AM
Ana Martínez
UX Designer
6/8 tareas


TeamMatch
Formación Inteligente de Equipos
Sistema Activo
Inicio
Mi Perfil
Proyectos
Mis Equipos
Evaluaciones
Evaluaciones
Feedback recibido y enviado
Dar Feedback
5.0
Rating Promedio
5.0
Colaboración
Excelente
4.0
Comunicación
Muy buena
5.0
Habilidades Técnicas
Experto
Feedback Recibido
Evaluaciones de tus compañeros de equipo (1)
CR
Carlos Rodríguez
SmartHealth - Plataforma de Telemedicina
5/5
7 abr 2026
Colaboración
5/5
Comunicación
4/5
Técnicas
5/5
"Excelente compañera de equipo, siempre dispuesta a ayudar y con gran dominio técnico"
Feedback Enviado


TeamMatch
Formación Inteligente de Equipos
Sistema Activo
Inicio
Mi Perfil
Proyectos
Mis Equipos
Evaluaciones
Excelente
Muy buena
Experto
Feedback Recibido
Evaluaciones de tus compañeros de equipo (1)
CR
Carlos Rodríguez
SmartHealth - Plataforma de Telemedicina
5/5
7 abr 2026
Colaboración
5/5
Comunicación
4/5
Técnicas
5/5
"Excelente compañera de equipo, siempre dispuesta a ayudar y con gran dominio técnico"
Feedback Enviado
Evaluaciones que has dado a tus compañeros (1)
AM
Ana Martínez
SmartHealth - Plataforma de Telemedicina
5/5
"Ana es increíble, sus diseños son siempre impecables y muy centrados en el usuario"